

The six architectural constraints for developing REST APIs

1. Client-server architecture

An API's job is to connect two pieces of software without limiting their own functionalities. This objective is one of the core restrictions of REST: the client (that makes requests) and the server (that gives responses) stay separate and independent.

When done properly, the client and server can update and evolve in different directions without having an impact on the quality of their data exchange. This is especially important in various cases where there are plenty of different clients a server has to cater to. Think about weather APIs — they have to send data to tons of different clients (all types of mobile devices are good examples) from a single database.

2. Statelessness

For an API to be stateless, it has to handle calls independently of each other. Each API call has to contain the data and commands necessary to complete the desired action.

An example of a non-stateless API would be if, during a session, only the first call has to contain the API key, which is then stored server-side. The following API calls depend on that first one since it provides the client's credentials.

In the same case, a stateless API will ensure that each call contains the API key and the server expects to see proof of access each time.

Stateless APIs have the advantage that one bad or failed call doesn't derail the ones that follow.

3. Uniform Interface

While the client and the server change in different ways, it's important that the API can still facilitate communication. To that end, REST APIs impose a uniform interface that can easily accommodate all connected software.

In most cases, that interface is based on the HTTP protocols. Besides the fact that it sets rules as to how the clients and the server may interact, it also has the advantage of being widely known and used on the Internet. Data is stored and exchanged through JSON files because of their versatility.

4. Layered system

To keep the API easy to understand and scale, RESTful architecture dictates that the design is structured into layers that operate together.

With a clear hierarchy for these layers, executing a command means that each layer does its function and then sends the data to the next one. Connected layers communicate with each other, but not with every component of the program. This way, the overall security of the API is also improved.

If the scope of the API changes, layers can be added, modified, or taken out without compromising other components of the interface.

5. Cacheability

It's not uncommon for a stateless API's requests to have large overhead. In some cases, that's unavoidable, but for repeated requests that need the same data, caching said information can make a huge difference.

The concept is simple: the client has the option to locally store certain pieces of data for a predetermined period of time. When they make a request for that data, instead of the server sending it again, they use the stored version.

The result is simple: instead of the client sending several difficult or costly requests in a short span of

time, they only have to do it once.

6. Code on Demand

Unlike the other constraints we talked about up to this point, the last one is optional. The reason for making “code on demand” optional is simple: it can be a large security risk.

The concept is to allow code or applets(now obsolete!) to be sent through the API and used for the application. As you can imagine, unknown code from a shady source could do some damage, so this constraint is best left for internal APIs where you have less to fear from hackers and people with bad intentions. Another drawback is that the code has to be in the appropriate programming language for the application, which isn’t always the case.

The upside is that “code on demand” can help the client implement their own features on the go, with less work being necessary on the API or server. In essence, it permits the whole system to be much more scalable and agile.